

Handling Dynamic Angular/PrimeNG Dropdowns

A step-by-step guide to locating and automating overlay-based dropdown components inside dynamic tables — with the Product Details grid as a working example.

Covers: DOM structure analysis, why framework-generated IDs fail, the `appendTo="body"` overlay pattern, xpath construction methodology, and working code in Selenium (Java & Python), Playwright, and Cypress.

Table of Contents

1. The Problem — What Makes This Element Hard to Automate
2. Reading the DOM: What Each Screenshot Tells Us
3. Root Cause #1 — Auto-Generated, Unstable IDs
4. Root Cause #2 — The appendTo="body" Overlay Pattern
5. Root Cause #3 — No Column Identity, Only Header Text
6. Step-by-Step: Building the Locator From Scratch
7. Full Working Code — Selenium (Java)
8. Full Working Code — Selenium (Python)
9. Full Working Code — Playwright (JavaScript)
10. Full Working Code — Cypress
11. Reusable Helper Pattern (Any Column, Any Row)
12. General Methodology Checklist for Any Overlay Widget
13. Common Pitfalls and How to Avoid Them
14. Recommendation to Development Team

1. The Problem

The screen in question is a **Commodity Contract Management — Create** form built with **Angular** and the **PrimeNG** UI component library. The "Product Details" grid is a dynamic, repeatable table (rows can be added/removed), and the **Product** column is rendered as a p-dropdown component rather than a plain HTML `<select>`.

This combination — *dynamic table + custom overlay dropdown* — is one of the most common sources of flaky, unmaintainable locators in UI test automation. Three separate problems stack on top of each other:

- The dropdown's DOM id is generated at runtime and changes between page loads.
- The list of options doesn't render where you clicked — it is teleported elsewhere in the DOM.
- The column itself has no name/id attribute; the only reliable identity is the header text "Product".

The rest of this guide walks through each problem individually, then builds a robust locator and automation script step by step.

2. Reading the DOM: What Each Screenshot Tells Us

2.1 — The rendered form

The Product Details grid shows columns: Product, Units, Packing, Qty, UOM, No Of Lots, Price, Target Type, Target Date Range, Trade Month, Position... Each of these is a distinct Angular component wrapped inside a table cell — dropdowns for Product/Packing/UOM/Target Type/Trade Month, plain inputs for Qty/Price, and a date-range picker for Target Date Range.

2.2 — The table header structure

Inspecting the header row reveals a structure like this (simplified):

```
<thead class="custom-card-header-table">
  <tr>
    <th style="display:none;"><span>Lot No.</span></th>
    <th><span>Product</span><span class="mandatory"> *</span></th>
    <th><span>Units</span><span class="mandatory"> *</span></th>
    <th><span>Packing</span><span class="mandatory"> *</span></th>
    ...
  </tr>
</thead>
```

Two details matter enormously here. First, the "Lot No." column is present in the DOM but hidden (`style="display:none;"`) — if you count columns naively by position without checking for hidden ones, your index will be off by one. Second, each header's text is split across two `<span>` tags — the label and the mandatory-field asterisk are separate nodes. A locator that does an exact text match on the whole `<th>` (e.g. `text = "Product"`) can fail; you need `normalize-space()` on the first span, or a `contains()` check.

2.3 — The dropdown trigger element

Expanding the Product cell shows the actual PrimeNG dropdown markup:

```
<p-dropdown appendto="body" class="p-element p-inputwrapper w-full ng-pristine ng-invalid ng-touched">
  <div id="pn_id_105" class="p-dropdown p-component p-inputwrapper p-dropdown-clearable">
    <span role="combobox" class="p-dropdown-label p-inputtext"
      aria-haspopup="listbox" aria-expanded="false" tabindex="0">
      <span>&nbsp;</span>
    </span>
    <div role="button" aria-label="dropdown trigger" aria-haspopup="listbox"
      class="p-dropdown-trigger" aria-expanded="false">
      <chevrondownicon class="p-icon-wrapper"></chevrondownicon>
    </div>
  </div>
</p-dropdown>
```

Three attributes are gold here: `role="combobox"` on the label span, `role="button"` with `aria-label="dropdown trigger"` on the clickable arrow, and `aria-haspopup="listbox"`. These ARIA attributes exist for accessibility, which means the framework is far less likely to change them than it is to change CSS class names or internal ids — making them much safer anchors for a locator.

2.4 — Chrome's auto-generated xpath

When you right-click → Copy → Copy XPath on the dropdown cell, DevTools produces something like:

```
//h3[normalize-space()='Product Details']/following::table[1]/tbody/tr[1]/td[
  count(
    //h3[normalize-space()='Product Details']/following::th[./span[normalize-space()='Product']]
    /preceding-sibling::th
  ) + 1
]
```

This is technically correct but fragile. It works by chaining three ideas: (1) find the section heading "Product Details", (2) find the first table that follows it, (3) compute which column position the "Product" header sits at, by counting how many `<th>` siblings come before it. We'll reuse this counting idea in Section 6, but rebuild it more defensively.

3. Root Cause #1 — Auto-Generated, Unstable IDs

In the screenshots, the dropdown container carries an id like `pn_id_105` or `pn_id_107`. This is **not** a static identifier assigned by the developer. PrimeNG maintains an internal counter and assigns `pn_id_<n>` to the *n*-th PrimeNG component instantiated during that page's lifecycle.

Why this breaks automation:

- Reloading the page can change the count if any component renders conditionally before this one.
- Adding a new row to the table increases the counter for every dropdown in that new row, but does *not* guarantee the same offset every time (conditional rendering, async data loads, etc. can shift it).
- The id is **consistent within a single page session** but should never be hardcoded into a reusable test — it will pass today and fail tomorrow with zero code changes on either side.

Rule of thumb:

Never write a locator that contains the literal string `"pn_id_"` followed by a number. Treat any id matching the pattern `id_[0-9]+`, `mat-[0-9]+`, or similar auto-increment patterns as disposable. Anchor on something a human reads on screen instead: section headings, column headers, labels, placeholder text, or ARIA attributes.

4. Root Cause #2 — The `appendTo="body"` Overlay Pattern

The `<p-dropdown appendto="body">` attribute is the single most important clue in this whole investigation. Here is what actually happens at runtime:

- **Before you click:** The dropdown's trigger (the visible box with the chevron) lives exactly where you'd expect — inside the table cell, inside the row, inside the Product Details table.
- **The moment you click it:** Angular's CDK/PrimeNG overlay mechanism dynamically creates a brand-new DOM node — typically a `<div class="p-dropdown-panel">` — and appends it as a direct child of `<body>`, completely outside the table's DOM subtree.
- **Why frameworks do this:** tables and grids often have `overflow: hidden` or `overflow-x: auto` (note the horizontal scrollbar visible under the Product Details grid). If the options panel rendered inside the cell, it would get clipped or scroll away with the table. Appending to `<body>` guarantees it renders on top of everything, positioned absolutely via JavaScript to sit visually under the trigger.

The practical consequence: **you cannot write a single xpath that both identifies the trigger and selects an option.** They are not ancestor/descendant of each other once the panel opens. You must treat this as a two-step interaction:

1) Locate + click the TRIGGER	-> scoped to the table/row/column
2) Locate + click the OPTION	-> scoped to the floating panel, searched anywhere in the DOM (usually the last child of <code><body></code>)

How to verify this yourself in DevTools:

Open the Elements panel, click the dropdown to open it, then press Ctrl+F (search) and type `p-dropdown-panel`. You'll see it listed as a sibling near the very end of `<body>`, not nested inside your table. This same pattern applies to PrimeNG's `p-multiselect`, `p-calendar`, `p-autocomplete`, and Angular Material's `mat-select` panels (via the CDK overlay) — so this technique generalizes far beyond just this one dropdown.

5. Root Cause #3 — No Column Identity, Only Header Text

Unlike a well-instrumented app, this grid's cells don't carry a `data-testid`, `name`, or `id` that says "this is the Product cell." The only thing that reliably identifies the column is the **visible header text**. That means any locator strategy must:

- Find the `<th>` whose text is "Product".
- Determine its position among sibling `<th>` elements (accounting for the hidden "Lot No." column).
- Apply that same numeric position to the `<td>` elements in the data row you care about.

This is a completely standard and reliable pattern in `xpath` — it's exactly what Chrome generated in Section 2.4 — but it is verbose. In Section 11 we turn it into a small reusable helper function so you only write it once.

6. Step-by-Step: Building the Locator From Scratch

STEP 1 | Scope to the correct table

Don't assume there's only one `<table>` on the page. Scope your search to the table that actually contains a "Product" header, so the locator still works even if the page later adds another grid above or below it.

```
//table[.//th[normalize-space(.)='Product']]
```

STEP 2 | Find the target header cell and its position

Count how many `<th>` siblings appear before the "Product" header, then add 1 to get its 1-based column index.

```
count(
  //table[.//th[normalize-space(.)='Product']]
  //th[normalize-space(.)='Product']/preceding-sibling::th
) + 1
```

STEP 3 | Select the row you need

For the first (or only) data row, use `tr[1]` inside `tbody`. For a specific row when the table has multiple lots, parameterize this index.

```
//table[.//th[normalize-space(.)='Product']]/tbody/tr[1]
```

STEP 4 | Combine into the full trigger locator

Putting it together, then drilling into the cell to the actual clickable `role="combobox"` element:

```
//table[.//th[normalize-space(.)='Product']]/tbody/tr[1]/td[
  count(//table[.//th[normalize-space(.)='Product']]/th[normalize-space(.)='Product']
    /preceding-sibling::th) + 1
]//span[@role='combobox']
```

STEP 5 | Locate the options panel after clicking

Once clicked, search the whole document for the open panel and the option by its visible text:

```
//div[contains(@class, 'p-dropdown-panel')]/li[normalize-space(.)='wheat']
```

Tip — confirm which panel is open:

If multiple dropdowns can be open across the app (they normally can't — PrimeNG closes others automatically), you can further filter using `[not(contains(@style, 'display: none'))]` or by checking `aria-expanded= 'true'` on the trigger before searching for the panel.

7. Full Working Code — Selenium (Java)

```
import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.*;
import java.time.Duration;

public class ProductDropdownHelper {

    private WebDriver driver;
    private WebDriverWait wait;

    public ProductDropdownHelper(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    // Builds an xpath for the trigger of ANY column identified by its header text
    private String triggerXpath(String columnHeader, int rowIndex) {
        return String.format(
            "//table[.//th[normalize-space(.)='%s']]tbody/tr[%d]/td[" +
            "count(//table[.//th[normalize-space(.)='%s']]//th[normalize-space(.)='%s']" +
            "/preceding-sibling::th) + 1]//span[@role='combobox']",
            columnHeader, rowIndex, columnHeader, columnHeader
        );
    }

    public void selectDropdownOption(String columnHeader, int rowIndex, String optionText) {
        // Step 1: click the trigger inside the table
        WebElement trigger = wait.until(ExpectedConditions.elementToBeClickable(
            By.xpath(triggerXpath(columnHeader, rowIndex))
        ));
        trigger.click();

        // Step 2: the panel is appended to <body> - search globally, not inside the table
        String optionXpath = String.format(
            "//div[contains(@class, 'p-dropdown-panel')]//li[normalize-space(.)='%s']",
            optionText
        );
        WebElement option = wait.until(ExpectedConditions.elementToBeClickable(
            By.xpath(optionXpath)
        ));
        option.click();
    }

    // Usage:
    // helper.selectDropdownOption("Product", 1, "Wheat");
    // helper.selectDropdownOption("Packing", 1, "Bags");
    // helper.selectDropdownOption("UOM", 1, "MT");
}
```

8. Full Working Code — Selenium (Python)

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class ProductDropdownHelper:
    def __init__(self, driver, timeout=10):
        self.driver = driver
        self.wait = WebDriverWait(driver, timeout)

    def _trigger_xpath(self, column_header, row_index=1):
        return (
            f"//table[.//th[normalize-space(.)='{column_header}']]tbody/tr[{row_index}]/td["
            f"count(//table[.//th[normalize-space(.)='{column_header}'])"
            f"//th[normalize-space(.)='{column_header}']/preceding-sibling::th) + 1"
            f"]//span[@role='combobox']"
        )

    def select_option(self, column_header, option_text, row_index=1):
        # Step 1: click the trigger scoped to the correct column + row
        trigger = self.wait.until(EC.element_to_be_clickable(
            (By.XPATH, self._trigger_xpath(column_header, row_index))
        ))
        trigger.click()

        # Step 2: the options list is appended to <body>; search the whole DOM
        option_xpath = (
            f"//div[contains(@class, 'p-dropdown-panel')]"
            f"//li[normalize-space(.)='{option_text}']"
        )
        option = self.wait.until(EC.element_to_be_clickable((By.XPATH, option_xpath)))
        option.click()

# Usage:
# helper = ProductDropdownHelper(driver)
# helper.select_option("Product", "Wheat")
# helper.select_option("Packing", "Bags")
# helper.select_option("UOM", "MT")
```

9. Full Working Code — Playwright (JavaScript)

Playwright's auto-waiting and CSS `:has()` support let this be written more compactly, and Playwright automatically waits for the floating panel to attach and become visible before interacting with it.

```
// helpers/dropdown.js
async function selectDropdownOption(page, columnHeader, optionText, rowIndex = 1) {
  // Step 1: scope to the table containing this header, then the nth data row,
  // then the cell at the matching column position, then the combobox trigger.
  const table = page.locator(`table:has(th:text-is("${columnHeader}"))`);
  const headerCells = table.locator("thead th");
  const count = await headerCells.count();

  let colIndex = -1;
  for (let i = 0; i < count; i++) {
    const text = (await headerCells.nth(i).innerText()).trim();
    if (text.startsWith(columnHeader)) { colIndex = i; break; }
  }
  if (colIndex === -1) throw new Error(`Column "${columnHeader}" not found`);

  const row = table.locator("tbody tr").nth(rowIndex - 1);
  const trigger = row.locator("td").nth(colIndex).locator("[role='combobox']");
  await trigger.click();

  // Step 2: panel is appended to <body> - search the whole page, not the table
  const option = page.locator(".p-dropdown-panel li", { hasText: optionText }).first();
  await option.waitFor({ state: "visible" });
  await option.click();
}

module.exports = { selectDropdownOption };

// Usage in a test:
// await selectDropdownOption(page, "Product", "Wheat");
// await selectDropdownOption(page, "Packing", "Bags");
```

10. Full Working Code — Cypress

Cypress cannot use xpath natively (without the cypress-xpath plugin), so the cleanest approach uses `cy.contains()` plus DOM traversal, which mirrors the same "find header, get index, apply to row" logic.

```
// cypress/support/dropdown.js
Cypress.Commands.add(
  "selectGridDropdown",
  (columnHeader, optionText, rowIndex = 1) => {
    cy.contains("table thead th", columnHeader)
      .then($th => {
        const table = $th.closest("table");
        const colIndex = Cypress.$(table).find("thead th").index($th);

        // Step 1: click the trigger in the correct row/column
        cy.wrap(table)
          .find("tbody tr")
          .eq(rowIndex - 1)
          .find("td")
          .eq(colIndex)
          .find("[role='combobox']")
          .click();
      });

    // Step 2: panel is appended to <body> - assert against the document root
    cy.get(".p-dropdown-panel li")
      .contains(optionText)
      .click();
  }
);

// Usage in a test:
// cy.selectGridDropdown("Product", "Wheat");
// cy.selectGridDropdown("Packing", "Bags");
```

11. Reusable Helper Pattern (Any Column, Any Row)

Notice that every implementation above follows the identical three-part shape. Once you build this helper for one dropdown, it works for **every** dropdown column in the grid (Product, Packing, UOM, Target Type, Trade Month, etc.) without modification — you're only changing the header text and option text arguments:

```
FUNCTION selectDropdown(columnHeaderText, optionText, rowIndex):

  1. table      = find table where any <th> text == columnHeaderText
  2. colPosition = index of that <th> among its siblings    (0-based or 1-based
                                                             depending on your tool)
  3. cell       = table -> tbody -> tr[rowIndex] -> td[colPosition]
  4. trigger    = cell -> descendant with role="combobox" (or aria-haspopup="listbox")
  5. click(trigger)
  6. panel      = anywhere in DOM: element with class containing "dropdown-panel"
                 (NOT inside `table` - it was moved to <body>)
  7. option     = panel -> descendant <li> (or [role="option"]) with text == optionText
  8. click(option)
```

For text inputs (Qty, Price) and the date-range picker (Target Date Range) in the same grid, steps 1–3 are identical — only step 4 onward differs, since those aren't overlay components. This is why it's worth investing in one well-tested "get cell by column header" utility and reusing it for every field type in the row.

12. General Methodology Checklist for Any Overlay Widget

Use this checklist any time you meet a new dynamic-table + custom-widget combination — it isn't specific to PrimeNG.

- **Never lock onto framework-generated IDs** such as `pn_id_*`, `mat-select-*`, `ng-*`, or any id that looks like a counter. Confirm by reloading the page — if the number changes, it's disposable.
- **Anchor on visible, human-readable text** — section headings, column headers, labels, placeholders, button text. These change far less often than markup internals.
- **Check for a teleported overlay** whenever a component can visually "float" above other content (dropdowns, date pickers, tooltips, modals, autocomplete lists). Search DevTools for `appendTo`, `cdk-overlay-container`, or `portal` in the class names.
- **Split trigger and panel into two locators** the moment you confirm an overlay pattern exists.
- **Compute column position dynamically** using `preceding-sibling::th` counts rather than hardcoding `td[3]` — column order can change when the UI is updated.
- **Watch for hidden columns** (`display:none`) thrown into the header row — they still count as siblings and will silently shift your index if you're not filtering for visibility.
- **Prefer ARIA roles/attributes** (`role="combobox"`, `aria-haspopup`, `aria-label`) over CSS classes, since accessibility attributes are more stable across framework/theme upgrades.
- **Always wait explicitly** for the panel to attach and become visible/clickable before interacting — overlay panels are inserted asynchronously via JavaScript, not present at initial page load.

13. Common Pitfalls and How to Avoid Them

Hardcoding pn_id_105 directly

Breaks the very next time the page renders a different number of PrimeNG components before it. Always derive the locator from header text instead.

Searching for the option inside the table's subtree

The panel isn't there — it was moved to via appendTo. The find will simply time out with "element not found", which is confusing because the option is visibly on screen.

Using td[3] with a fixed index

Silently selects the wrong field if a column is added/removed/reordered, or if a hidden column (like Lot No.) is present but not accounted for.

Clicking the trigger too fast after a previous panel closed

PrimeNG animates panel close/open; clicking mid-animation can register on the wrong element. Add an explicit wait for aria-expanded to flip, or for the previous panel to detach from the DOM.

Matching option text with exact equality when there's extra whitespace

PrimeNG often wraps option text in its own , sometimes with leading/trailing whitespace or . Always use normalize-space() (xpath) or trimmed text comparisons (JS) rather than raw string equality.

14. Recommendation to the Development Team

Everything in this guide is a workaround for the absence of stable test hooks. The most effective long-term fix costs the dev team very little and removes nearly all of this complexity:

- Add a `data-testid` (or similar) attribute to each dropdown wrapper, e.g. `data-testid="product-dropdown-row-{index}"`.
- Add matching `data-testid` attributes to each rendered `<li>` option, e.g. `data-testid="option-wheat"`.
- If using PrimeNG's `appendTo`, consider whether `appendTo="self"` is viable in this particular grid (no overflow clipping) — it keeps the panel nested with the trigger and removes the two-step locator entirely.

With those two attributes in place, every one of the locators in this guide collapses to a single line, e.g. `page.locator('[data-testid="product-dropdown-row-1"]')`, immune to column reordering, header text changes, and PrimeNG id counters.

End of guide.